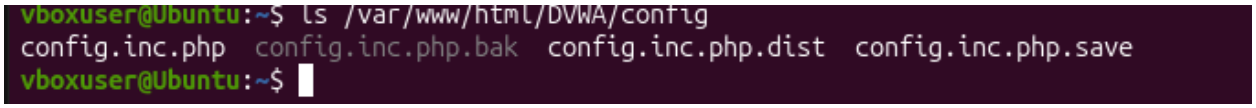
Check if DAWA is installed in Ubuntu



Let's confirm the configuration file

ls /var/www/html/DVWA/config

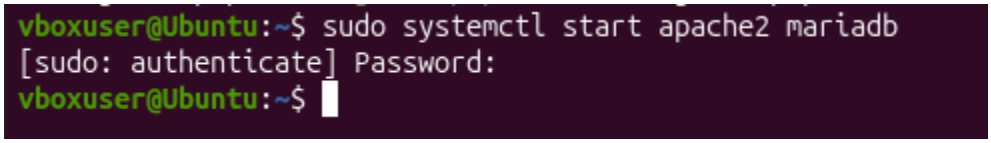


Your config file is there — `config.inc.php` is the first one listed. (The `.dist` is the original template, and `.bak` / `.save` are backup copies from when you were editing it earlier. Only `config.inc.php` is the one that's actually used; the others are harmless leftovers you can ignore.)

So my earlier setup work is fully intact: DVWA downloaded, config file in place with my database settings. We've passed all the file setup.

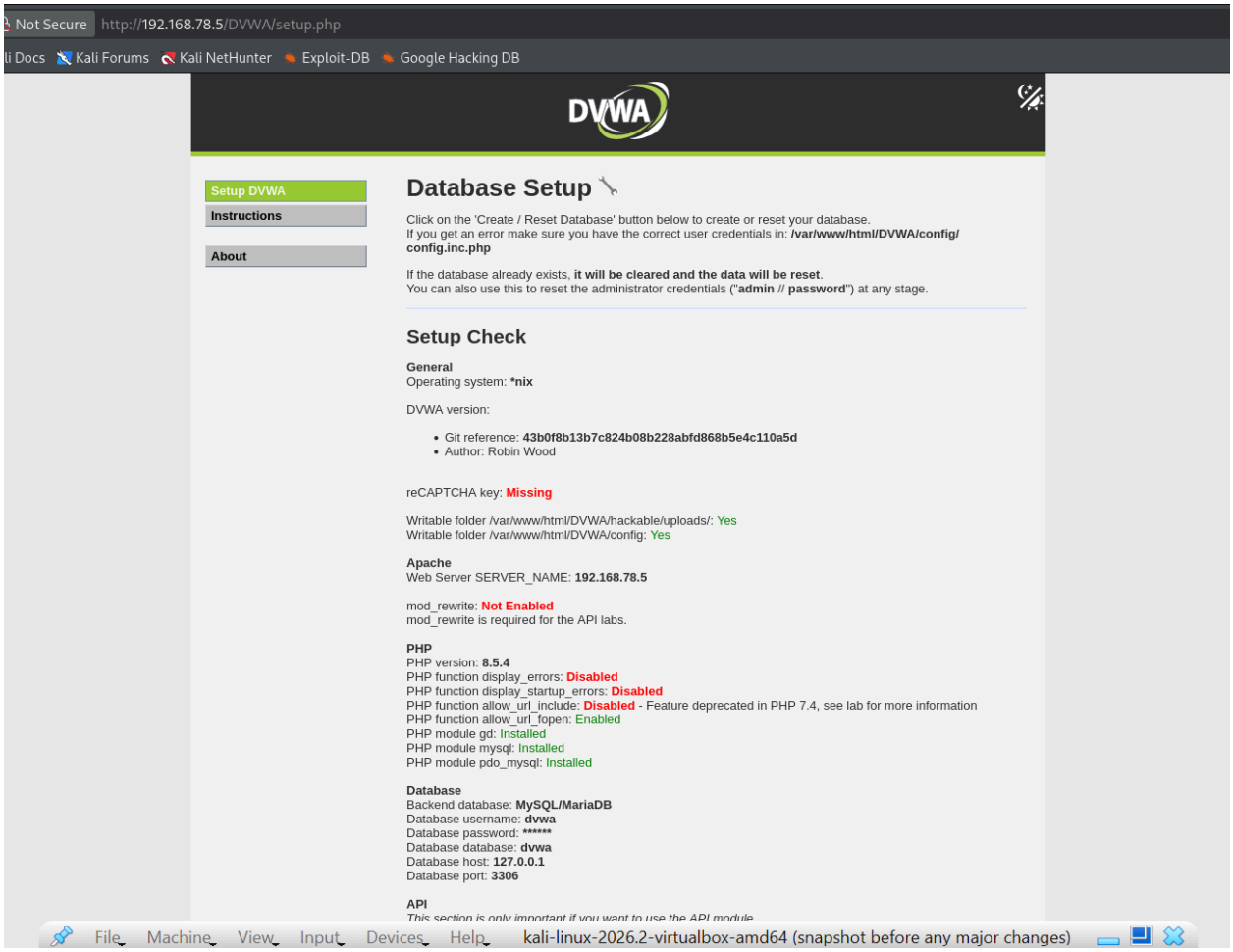
Now, let's just bring it online. On **Ubuntu**, we need to make sure the services are running:

sudo systemctl start apache2 mariadb



Then on **Kali**, open Firefox and go to:

http://192.168.78.5/DVWA/setup.php



the DVWA setup page when opened it in Firefox did download nicely. I deliberately installed PostgreSQL on the Ubuntu target. And I think it is worth chasing port 5432 another minute.

`allow_url_include: Disabled` — needed only for the **Remote File Inclusion (RFI)** lab. If you plan to demonstrate file inclusion as one of your OWASP findings, turn it on; otherwise you can leave it. To enable:

sudo nano /etc/php/8.5/apache2/php.ini

Find `allow_url_include` and set it to `On`, save (Ctrl+O, Enter, Ctrl+X), then:

sudo systemctl restart apache2

`mod_rewrite: Not Enabled` — only needed for the **API labs**, which you almost certainly don't need for this rubric. Safe to ignore unless your course specifically wants the API module.

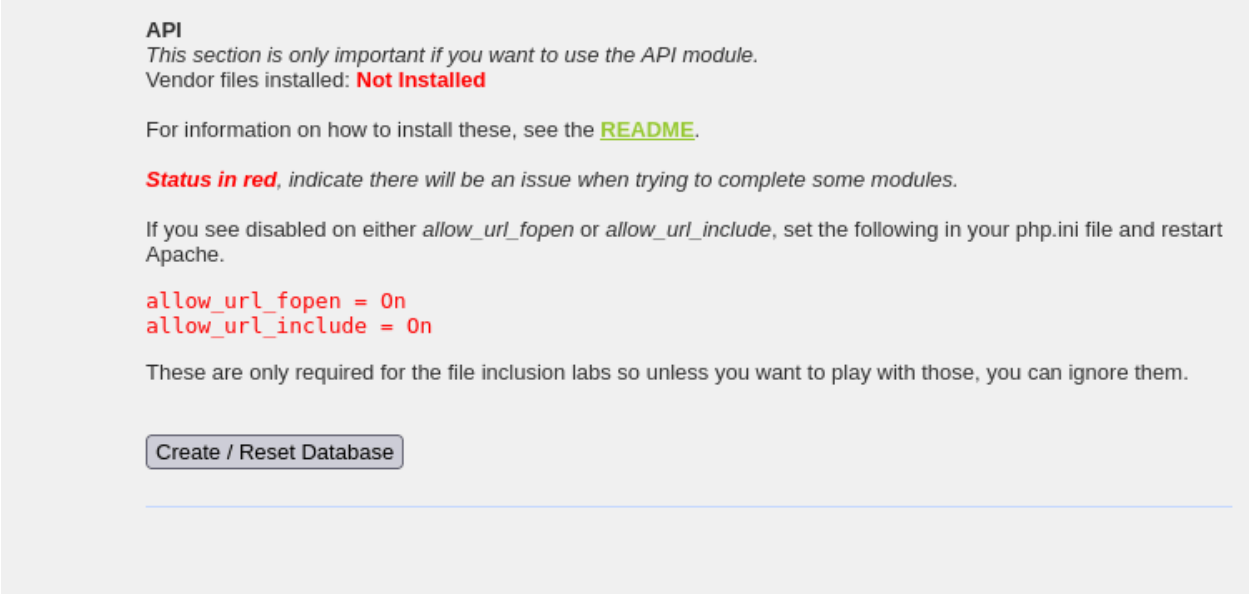
Neither of these blocks the main vulnerabilities you care about (SQL injection, XSS, command injection, CSRF) — those work regardless.

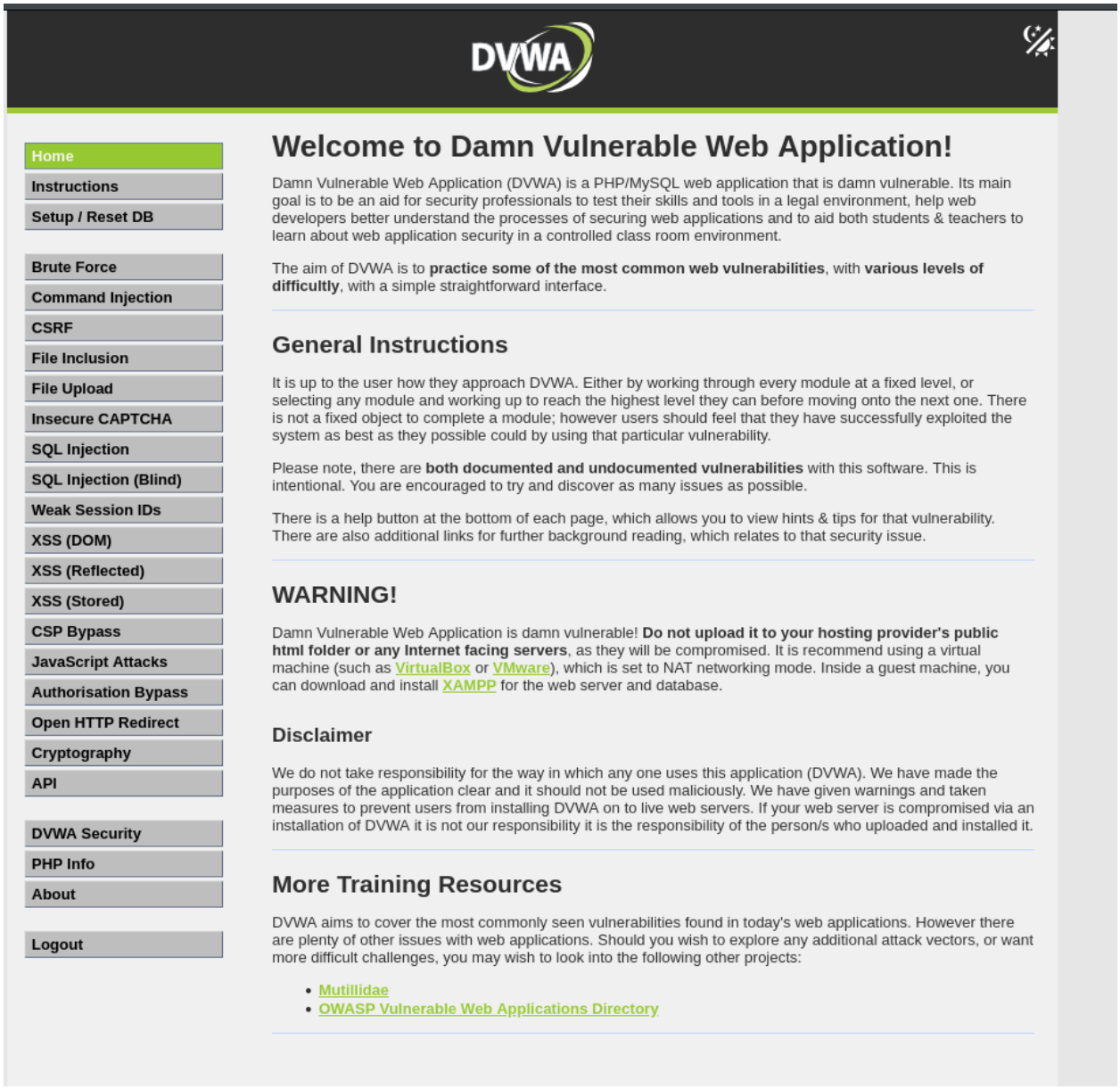
Your next action:

Scroll to the very bottom of this page and click the **Create / Reset Database** button. That builds the `dvwa` database tables using the user credentials it just showed. Since all your database settings match, it should succeed and redirect you to the login page.

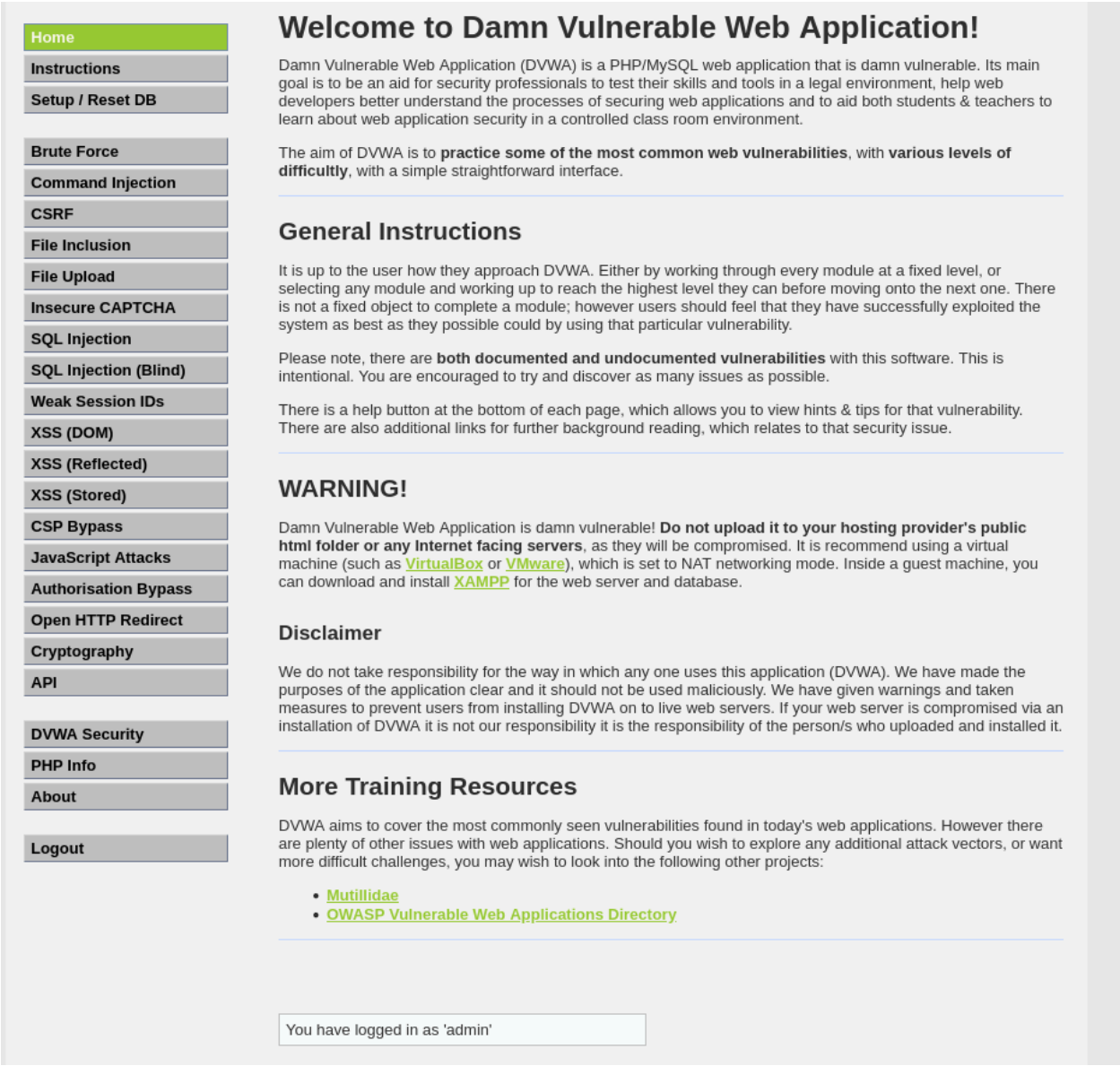
Then log in with:

- Username: `admin`
- Password: `password`





Now, let’s Create / Reset Database, then try logging in. Tell me whether you reach the DVWA main page — and if anything turns red when you click the button, paste it and we'll sort it.



This is the finish line for your DVWA install. I logged in as `admin`, the full menu of vulnerability modules is there on the left — SQL Injection, XSS (three variants), Command Injection, CSRF, File Inclusion, File Upload, and more — and the bottom confirms **"I have logged in as 'admin'."** DVWA is fully installed and working.

One thing to do before you start testing: click **DVWA Security** in the left menu and set the level to **Low**, then Submit. By default it's often set to `impossible`, where every vulnerability is patched — at that level your scans and manual tests will find nothing. Low is where the vulnerabilities are actually exploitable, which is what you need for the OWASP findings.

Here's the natural next sequence:

1. Scope the tools to this one target. In ZAP, create a Context and set the scope to `http://192.168.78.5/DVWA/`, excluding everything else. In Burp, add the same to Target → Scope. Screenshot both — this is your documented-boundaries evidence the rubric asks for.

2. Run an automated scan with ZAP against the DVWA URL and let it report what it finds.

3. Manually confirm a few of the big ones with Burp — intercept a request, then use Repeater to test something like SQL Injection or XSS by hand, so you have both automated and manual evidence.

4. Document each finding with: which OWASP category it falls under, a screenshot of the proof, and a remediation recommendation.

Want to start with setting the security level and then move straight into scoping ZAP, or would you rather tackle one specific vulnerability (like SQL Injection) end-to-end first as a worked example?